

Folha de Dicas

Pacote/Método	Descrição	Exemplo de código
NLTK	NLTK é uma biblioteca Python usada em processamento de linguagem natural (NLP) para tarefas como tokenização e processamento de texto. O exemplo de código mostra como você pode tokenizar texto usando o tokenizador baseado em palavras do NLTK.	<pre>import nltk nltk.download("punkt") from nltk.tokenize import word_tokenize text = "Unicórnios são reais. Eu vi um unicórnio ontem. Não consegui vê-lo hoje." token = word_tokenize(text) print(token)</pre>
spaCy	spaCy é uma biblioteca de código aberto usada em NLP. Ela fornece ferramentas para tarefas como tokenização e embeddings de palavras. O exemplo de código mostra como você pode tokenizar texto usando o tokenizador baseado em palavras do spaCy.	<pre>import spacy text = "Unicórnios são reais. Eu vi um unicórnio ontem. Não consegui vê-lo hoje." nlp = spacy.load("en_core_web_sm") doc = nlp(text) token_list = [token.text for token in doc] print("Tokens:", token_list)</pre>
BertTokenizer	BertTokenizer é um tokenizador baseado em subpalavras que usa o algoritmo WordPiece. O exemplo de código mostra como você pode tokenizar texto usando o BertTokenizer.	<pre>from transformers import BertTokenizer tokenizer = BertTokenizer.from_pretrained("bert-base-uncased") tokenizer.tokenize("A IBM me ensinou sobre tokenização.")</pre>
XLNetTokenizer	XLNetTokenizer tokeniza texto usando os algoritmos Unigram e SentencePiece. O exemplo de código mostra como você pode tokenizar texto usando o XLNetTokenizer.	<pre>from transformers import XLNetTokenizer tokenizer = XLNetTokenizer.from_pretrained("xlnet-base-cased") tokenizer.tokenize("A IBM me ensinou sobre tokenização.")</pre>

Pacote/Método	Descrição	Exemplo de código
torchtext	A biblioteca torchtext é parte do ecossistema PyTorch e fornece as ferramentas e funcionalidades necessárias para NLP. O exemplo de código mostra como você pode usar o torchtext para gerar tokens e convertê-los em índices.	<pre>from torchtext.vocab import build_vocab_from_iterator</pre> Define um conjunto de dados <pre>dataset = [(1,"Introdução ao NLP"), (2,"Fundamentos do PyTorch"), (1,"Técnicas de NLP para Classificação de Texto"), (3,"Reconhecimento de Entidades Nomeadas com PyTorch"), (3,"Análise de Sentimentos usando PyTorch"), (3,"Tradução Automática com PyTorch"), (1,"NLP Entidade Nomeada, Análise de Sentimentos, Tradução Automática"), (1,"Tradução Automática com NLP"), (1,"Entidade Nomeada vs Análise de Sentimentos em NLP")]</pre> Aplica o tokenizador ao texto para obter os tokens com <pre>from torchtext.data.utils import get_tokenizer tokenizer = get_tokenizer("basic_english") tokenizer(dataset[0][1])</pre> Recebe um iterador de dados como entrada, processa o t e gera a saída tokenizada individualmente <pre>def yield_tokens(data_iter): for _,text in data_iter: yield tokenizer(text)</pre> Cria um iterador <pre>my_iterator = yield_tokens(dataset)</pre> Busca o próximo conjunto de tokens do conjunto de dado <pre>next(my_iterator)</pre> Converte tokens em índices e define <unk> como o palavra padrão se uma palavra não for encontrada no vc <pre>vocab = build_vocab_from_iterator(yield_tokens(dataset), specials=["<unk>"]) vocab.set_default_index(vocab["<unk>"])</pre> Fornece um dicionário que mapeia palavras para seus ín <pre>vocab.get_stoi()</pre>
vocab	O objeto vocab faz parte da biblioteca torchtext do PyTorch. Ele mapeia tokens para índices. O exemplo de código mostra como você pode aplicar o objeto vocab diretamente aos tokens.	<pre># Recebe um iterador como entrada e extrai a próxima frase tokenizada.</pre> Cria uma lista de índices de tokens usando o dicionári <pre>def get_tokenized_sentence_and_indices(iterator): tokenized_sentence = next(iterator) token_indices = [vocab[token] for token in tokenized_sentence] return tokenized_sentence, token_indices</pre> Retorna as sentenças tokenizadas e os índices de token Repete o processo. <pre>tokenized_sentence, token_indices = get_tokenized_sentence_and_indices(meu_iterador) next(meu_iterador)</pre> Imprime a sentença tokenizada e seus índices de token <pre>print("Sentença Tokenizada:", tokenized_sentence) print("Índices de Token:", token_indices)</pre>

Pacote/Método	Descrição	Exemplo de código
Tokens especiais no PyTorch: <eos> e <bos>	Tokens especiais são tokens introduzidos nas sequências de entrada para transmitir informações específicas ou servir a um propósito particular durante o treinamento. O token <bos> denota o início da sequência de entrada, e o token <eos> denota o fim.	<pre># Adiciona <bos> no início e <eos> no final das sentenças tokenizadas</pre> usando um loop que itera sobre as sentenças nos dados <pre>tokenizer_en = get_tokenizer('spacy', language='en_core_web_sm') tokens = [] max_length = 0 for line in lines: tokenized_line = tokenizer_en(line) tokenized_line = ['<bos>'] + tokenized_line + ['<eos>'] tokens.append(tokenized_line) max_length = max(max_length, len(tokenized_line))</pre>
Tokens especiais no PyTorch: <pad>	O exemplo de código mostra o uso do token <pad> para garantir que todas as sentenças tenham o mesmo comprimento.	<pre># Preenche as linhas tokenizadas for i in range(len(tokens)): tokens[i] = tokens[i] + ['<pad>'] * (max_length - len(tokens[i]))</pre>
Classe Dataset no PyTorch	A classe Dataset permite acessar e recuperar amostras individuais de um conjunto de dados. O exemplo de código mostra como você pode criar um conjunto de dados personalizado e acessar amostras.	<pre># Importa a classe Dataset e define uma lista de sentenças from torch.utils.data import Dataset sentences = ["Se você quer saber como é um homem, observe como ele trata seus inferiores, não seus iguais.", "Fae é uma amiga inconstante, Harry."] Faz o download e lê os dados class CustomDataset(Dataset): def init(self, sentences): self.sentences = sentences # Retorna o comprimento dos dados def len(self): return len(self.sentences) # Retorna um item no índice def getitem(self, idx): return self.sentences[idx] Cria um objeto de conjunto de dados dataset=CustomDataset(sentences) Acessos a amostras como em uma lista Por exemplo, dataset[0]</pre>
Classe DataLoader no PyTorch	A classe DataLoader permite o	<pre># Cria um objeto iterador data_iter = iter(dataloader)</pre>

Pacote/Método	Descrição	Exemplo de código
	carregamento e a iteração eficientes sobre conjuntos de dados para treinar modelos de aprendizado profundo. O exemplo de código mostra como você pode usar a classe DataLoader para gerar lotes de sentenças para processamento adicional, como treinar um modelo de rede neural	Chama a próxima função para retornar novos lotes de ar <pre>next(data_iter)</pre> Cria uma instância do conjunto de dados personalizado <pre>from torch.utils.data import DataLoader custom_dataset = CustomDataset(sentences)</pre> Especifica um tamanho de lote <pre>batch_size = 2</pre> Cria um carregador de dados <pre>dataloader = DataLoader(custom_dataset, batch_size=batch_size, shuffle=True)</pre> Imprime as sentenças em cada lote <pre>for batch in dataloader: print(batch)</pre>
Função de colagem personalizada no PyTorch	A função de colagem personalizada é uma função definida pelo usuário que define como amostras individuais são coladas ou agrupadas. Você pode utilizar a função de colagem para tarefas como tokenização, conversão de índices tokenizados e transformação do resultado em um tensor. O exemplo de código mostra como você pode usar uma função de colagem personalizada em um carregador de dados.	<pre># Define uma função de colagem personalizada def collate_fn(batch): tensor_batch = []</pre> Tokeniza cada amostra no lote <pre>for sample in batch: tokens = tokenizer(sample)</pre> Mapeia tokens para números usando o vocab <pre>tensor_batch.append(torch.tensor([vocab[token] for token in tokens]))</pre> Preenche as sequências dentro do lote para ter compr <pre>padded_batch = pad_sequence(tensor_batch, batch_first=True) return padded_batch</pre> Cria um carregador de dados usando a função de colagem <pre>dataloader = DataLoader(custom_dataset, batch_size=batch_size, shuffle=True, collate_fn=collate_fn)</pre>



Skills Network